

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
import numpy as np
import os
from google.colab import files
import pdfplumber
import pytesseract
from PIL import Image
import re
from io import BytesIO
```

```
#
```

```
—
```

```
# PALETA DE CORES
```

```
#
```

```
—
```

```
COR_SOL      = "#FFB300"
COR_VERDE    = "#2ECC71"
COR_AZUL     = "#1A73E8"
COR_CINZA    = "#BDC3C7"
COR_FUNDO    = "#0D1117"
COR_PAINEL   = "#161B22"
COR_TEXTO    = "#E6EDF3"
COR_ACENTO   = "#F97316"
COR_VERMELHO = "#E74C3C"
```

```
#
```

```
—
```

```
# 1. ENTRADAS DO USUÁRIO
```

```
#
```

```
—
```

```
def solicitar_dados():
    print("\n" + "=" * 60)
    print(" 🌞 SISTEMA DE ANÁLISE SOLAR FOTOVOLTAICA 🌞")
    print("=" * 60)

    consumo_mensal_kwh = float(input("\n📊 Consumo médio mensal (kWh): "))
    tarifa_kwh          = float(input("💰 Tarifa de energia (R$/kWh) [ex: 0.85]: "))
```

```
horas_sol_dia    = float(input("☀ Média de horas de sol pleno/dia [ex: 5.0]: "))
anos_simulacao   = int(input("📅 Anos de simulação [ex: 25]: "))
```

```
return consumo_mensal_kwh, tarifa_kwh, horas_sol_dia, anos_simulacao
```

```
#
```

``` # 2. CÁLCULOS PRINCIPAIS ```

```
#
```

```
def calcular_sistema(consumo_mensal_kwh, tarifa_kwh, horas_sol_dia, anos_simulacao):
```

```
    # Potência necessária (kWp)
```

```
    consumo_diario_kwh = consumo_mensal_kwh / 30
```

```
    eficiencia_sistema = 0.80      # perdas típicas: inversor, cabos, temperatura
```

```
    potencia_kwp       = consumo_diario_kwh / (horas_sol_dia * eficiencia_sistema)
```

```
    # Número de painéis (painel padrão 450 Wp)
```

```
    potencia_painel_kw = 0.450
```

```
    num_paneis         = int(np.ceil(potencia_kwp / potencia_painel_kw))
```

```
    potencia_real_kwp  = num_paneis * potencia_painel_kw
```

```
    # Custo do investimento (R$)
```

```
    custo_por_kwp      = 4_500.00    # média mercado BR 2024 (instalação inclusa)
```

```
    custo_total        = potencia_real_kwp * custo_por_kwp
```

```
    # Geração mensal real (kWh)
```

```
    geracao_mensal_kwh = potencia_real_kwp * horas_sol_dia * 30 * eficiencia_sistema
```

```
    # Economia mensal (R$)
```

```
    economia_mensal    = geracao_mensal_kwh * tarifa_kwh
```

```
    # Reajuste médio anual da tarifa
```

```
    reajuste_anual     = 0.06        # 6% ao ano (histórico ANEEL)
```

```
    # — Simulação mês a mês —
```

```
    meses_total        = anos_simulacao * 12
```

```
    acumulado_economias = []
```

```
    economias_mensais   = []
```

```
    saldo_acumulado    = []
```

```
payback_mes      = None
```

```
tarifa_atual     = tarifa_kwh
```

```
acumulado        = 0.0
```

```
for mes in range(1, meses_total + 1):
```

```
    if mes % 12 == 1 and mes > 1:
```

```
        tarifa_atual *= (1 + reajuste_anual)
```

```
    economia_mes   = geracao_mensal_kwh * tarifa_atual
```

```
    acumulado      += economia_mes
```

```
    saldo          = acumulado - custo_total
```

```
    economias_mensais.append(economia_mes)
```

```
    acumulado_economias.append(acumulado)
```

```
    saldo_acumulado.append(saldo)
```

```
    if saldo >= 0 and payback_mes is None:
```

```
        payback_mes = mes
```

```
payback_anos = payback_mes / 12 if payback_mes else None
```

```
lucro_total  = acumulado_economias[-1] - custo_total
```

```
roi_pct      = (lucro_total / custo_total) * 100
```

```
return {
```

```
    "consumo_mensal_kwh" : consumo_mensal_kwh,
```

```
    "tarifa_kwh"         : tarifa_kwh,
```

```
    "horas_sol_dia"      : horas_sol_dia,
```

```
    "potencia_kwp"       : potencia_kwp,
```

```
    "potencia_real_kwp"  : potencia_real_kwp,
```

```
    "num_paineis"        : num_paineis,
```

```
    "custo_total"        : custo_total,
```

```
    "geracao_mensal_kwh" : geracao_mensal_kwh,
```

```
    "economia_mensal"    : economia_mensal,
```

```
    "payback_mes"        : payback_mes,
```

```
    "payback_anos"       : payback_anos,
```

```
    "acumulado_economias" : acumulado_economias,
```

```
    "economias_mensais"  : economias_mensais,
```

```
    "saldo_acumulado"    : saldo_acumulado,
```

```
    "lucro_total"        : lucro_total,
```

```
    "roi_pct"            : roi_pct,
```

```
    "anos_simulacao"     : anos_simulacao,
```

```
    "meses_total"        : meses_total,
```

```
    "reajuste_anual" : reajuste_anual,  
    "eficiencia_sistema" : eficiencia_sistema,  
}
```

```
#
```

```
# 3. RELATÓRIO NO TERMINAL
```

```
#
```

```
def exibir_relatorio(r):
```

```
    print("\n" + "=" * 60)
```

```
    print(" 📋 RELATÓRIO DE VIABILIDADE")
```

```
    print("=" * 60)
```

```
    print(f"\n 🏠 SISTEMA DIMENSIONADO")
```

```
    print(f"   Potência necessária : {r['potencia_kwp']:.2f} kWp")
```

```
    print(f"   Potência instalada : {r['potencia_real_kwp']:.2f} kWp")
```

```
    print(f"   Nº de painéis (450W) : {r['num_paineis']} unidades")
```

```
    print(f"\n 💰 FINANCEIRO")
```

```
    print(f"   Investimento total : R$ {r['custo_total']:.2f}")
```

```
    print(f"   Geração mensal : {r['geracao_mensal_kwh']:.1f} kWh")
```

```
    print(f"   Economia 1º mês : R$ {r['economia_mensal']:.2f}")
```

```
    print(f"   Reajuste tarifário : {r['reajuste_anual']*100:.0f}% ao ano (projetado)")
```

```
    print(f"\n ⌚ RETORNO DO INVESTIMENTO")
```

```
    if r['payback_anos']:
```

```
        print(f"   Payback : {r['payback_mes']} meses "
```

```
              f"({r['payback_anos']:.1f} anos)")
```

```
    else:
```

```
        print(f"   Payback : não atingido no período simulado")
```

```
    print(f"\n 🏆 RESULTADO EM {r['anos_simulacao']} ANOS")
```

```
    print(f"   Total economizado : R$ {r['acumulado_economias'][-1]:.2f}")
```

```
    print(f"   Lucro líquido : R$ {r['lucro_total']:.2f}")
```

```
    print(f"   ROI : {r['roi_pct']:.1f}%")
```

```
    print(f"   CO2 evitado (~) : {r['geracao_mensal_kwh'] * r['meses_total'] * 0.0817:.0f} kg")
```

```
    if r['payback_anos'] and r['payback_anos'] <= r['anos_simulacao']:
```

```
        print(f"\n ✅ VIÁVEL – o investimento se paga em {r['payback_anos']:.1f} anos!")
```

```

else:
    print(f"\n ⚠️ ATENÇÃO – payback não atingido no período analisado.")
    print("=" * 60 + "\n")

```

```
#
```

```
—
```

```
# 4. GRÁFICOS
```

```
#
```

```
—
```

```

def gerar_graficos(r):
    plt.rcParams.update({
        "font.family" : "DejaVu Sans",
        "axes.facecolor" : COR_PAINEL,
        "figure.facecolor" : COR_FUNDO,
        "text.color" : COR_TEXTO,
        "axes.labelcolor" : COR_TEXTO,
        "xtick.color" : COR_TEXTO,
        "ytick.color" : COR_TEXTO,
        "axes.edgecolor" : "#30363D",
        "grid.color" : "#21262D",
        "grid.linestyle" : "--",
        "grid.alpha" : 0.6,
    })

    meses = np.arange(1, r["meses_total"] + 1)
    anos_eixo = meses / 12

    fig = plt.figure(figsize=(18, 14))
    fig.suptitle("☀️ Análise de Viabilidade – Sistema Solar Fotovoltaico",
                fontsize=20, fontweight="bold", color=COR_SOL, y=0.98)

    gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.38)

    # — GRÁFICO 1: Saldo acumulado (payback) —————
    ax1 = fig.add_subplot(gs[0, :2])
    ax1.set_title("Saldo Acumulado × Investimento (R$)", color=COR_TEXTO, fontsize=13,
                  pad=10)

    saldo = np.array(r["saldo_acumulado"])
    positivo = np.where(saldo >= 0, saldo, np.nan)

```

```

negativo = np.where(saldo < 0, saldo, np.nan)

ax1.fill_between(anos_eixo, negativo, 0, color=COR_VERMELHO, alpha=0.25,
label="Prejuízo")
ax1.fill_between(anos_eixo, positivo, 0, color=COR_VERDE, alpha=0.25, label="Lucro")
ax1.plot(anos_eixo, saldo, color=COR_VERDE, linewidth=2.2)
ax1.axhline(0, color=COR_CINZA, linewidth=1.2, linestyle="--")

if r["payback_anos"]:
    ax1.axvline(r["payback_anos"], color=COR_SOL, linewidth=2, linestyle=":",
label=f"Payback: {r['payback_anos']:.1f} anos")
    ax1.annotate(f" Payback\n {r['payback_anos']:.1f} anos",
xy=(r["payback_anos"], 0),
xytext=(r["payback_anos"] + 0.5, r["lucro_total"] * 0.15),
color=COR_SOL, fontsize=10, fontweight="bold",
arrowprops=dict(arrowstyle="->", color=COR_SOL, lw=1.4))

ax1.yaxis.set_major_formatter(
plt.FuncFormatter(lambda x, _: f"R$ {x/1000:.0f}k" if abs(x) >= 1000 else f"R$ {x:.0f}"))
ax1.set_xlabel("Anos")
ax1.set_ylabel("Saldo (R$)")
ax1.legend(facecolor=COR_FUNDO, edgecolor="#30363D", labelcolor=COR_TEXTO,
fontsize=9)
ax1.grid(True)

# — GRÁFICO 2: KPI Card —————
ax2 = fig.add_subplot(gs[0, 2])
ax2.set_facecolor(COR_PAINEL)
ax2.axis("off")

kpis = [
    ("Investimento", f"R$ {r['custo_total']:.0f}", COR_ACENTO),
    ("Payback", f"{r['payback_anos']:.1f} anos" if r["payback_anos"] else "N/A", COR_SOL),
    ("ROI total", f"{r['roi_pct']:.0f}%", COR_VERDE),
    ("Nº painéis", str(r["num_paineis"])),
COR_AZUL),
    ("Potência inst.", f"{r['potencia_real_kwp']:.2f} kWp", COR_CINZA),
]

ax2.set_xlim(0, 1); ax2.set_ylim(0, 1)
for i, (label, valor, cor) in enumerate(kpis):
    y = 0.88 - i * 0.18
    ax2.text(0.05, y, label, fontsize=9, color=COR_CINZA, va="center")

```

```

ax2.text(0.05, y - 0.07, valor, fontsize=14, color=cor, va="center", fontweight="bold")

ax2.set_title("Indicadores-chave", color=COR_TEXTO, fontsize=11, pad=8)

# — GRÁFICO 3: Economia mensal ao longo do tempo —————
ax3 = fig.add_subplot(gs[1, :2])
ax3.set_title("Economia Mensal ao Longo do Tempo (R$)", color=COR_TEXTO, fontsize=13,
pad=10)

ax3.bar(anos_eixo, r["economias_mensais"], width=0.07,
        color=COR_AZUL, alpha=0.7, label="Economia / mês")
ax3.plot(anos_eixo, r["economias_mensais"], color=COR_SOL,
        linewidth=1.5, alpha=0.9, label="Tendência (reajuste anual)")

ax3.yaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _ : f"R$ {x:.0f}"))
ax3.set_xlabel("Anos")
ax3.set_ylabel("Economia (R$)")
ax3.legend(facecolor=COR_FUNDO, edgecolor="#30363D", labelcolor=COR_TEXTO,
fontsize=9)
ax3.grid(True, axis="y")

# — GRÁFICO 4: Pizza – composição do custo —————
ax4 = fig.add_subplot(gs[1, 2])
ax4.set_title("Composição do Custo\ndo Sistema", color=COR_TEXTO, fontsize=11, pad=8)

labels_pizza = ["Painéis\nFotovoltaicos", "Inversor", "Instalação\n& Estrutura",
                "Cabeamento\n& Outros"]
sizes = [45, 20, 25, 10]
cores_p = [COR_SOL, COR_AZUL, COR_ACENTO, COR_CINZA]
explode = [0.04] * 4

wedges, texts, autotexts = ax4.pie(
    sizes, labels=labels_pizza, autopct="%1.0f%%",
    colors=cores_p, explode=explode,
    textprops={"color": COR_TEXTO, "fontsize": 8},
    wedgeprops={"edgecolor": COR_FUNDO, "linewidth": 1.5})
for at in autotexts:
    at.set_color(COR_FUNDO)
    at.set_fontweight("bold")

# — GRÁFICO 5: Geração vs Consumo (barras mensais – ano 1) —
ax5 = fig.add_subplot(gs[2, :2])

```

```

ax5.set_title("Geração Solar × Consumo – Variação Mensal Estimada (Ano 1)",
              color=COR_TEXTO, fontsize=13, pad=10)

meses_nome = ["Jan", "Fev", "Mar", "Abr", "Mai", "Jun",
              "Jul", "Ago", "Set", "Out", "Nov", "Dez"]

# Fator de irradiação por mês (relativo ao valor informado)
fator_irrad = np.array([1.15, 1.10, 1.05, 0.95, 0.85, 0.80,
                        0.82, 0.88, 0.95, 1.05, 1.10, 1.12])

geracao_mensal_variavel = r["geracao_mensal_kwh"] * fator_irrad
consumo_variavel = r["consumo_mensal_kwh"] * np.array(
    [1.0, 1.0, 1.02, 1.05, 1.10, 1.15, 1.18, 1.15, 1.08, 1.03, 1.01, 1.0])

x = np.arange(12)
larg = 0.35

ax5.bar(x - larg/2, geracao_mensal_variavel, larg,
        label="Geração (kWh)", color=COR_SOL, alpha=0.85)
ax5.bar(x + larg/2, consumo_variavel, larg,
        label="Consumo (kWh)", color=COR_AZUL, alpha=0.85)

ax5.set_xticks(x)
ax5.set_xticklabels(meses_nome, color=COR_TEXTO, fontsize=9)
ax5.set_ylabel("kWh")
ax5.legend(facecolor=COR_FUNDO, edgecolor="#30363D", labelcolor=COR_TEXTO,
           fontsize=9)
ax5.grid(True, axis="y")

# — GRÁFICO 6: Economia acumulada vs custo —————
ax6 = fig.add_subplot(gs[2, 2])
ax6.set_title("Acumulado\nvs Investimento", color=COR_TEXTO, fontsize=11, pad=8)

anos_marco = [5, 10, 15, 20, 25]
economias_m = []
for a in anos_marco:
    mes_idx = min(a * 12 - 1, r["meses_total"] - 1)
    economias_m.append(r["acumulado_economias"][mes_idx])

cores_bar = []
for e in economias_m:
    cores_bar.append(COR_VERDE if e >= r["custo_total"] else COR_VERMELHO)

```



```

bars = ax6.bar([str(a) + "a" for a in anos_marco],
               economias_m, color=cores_bar, alpha=0.85, edgecolor=COR_FUNDO,
linewidth=1.2)
ax6.axhline(r["custo_total"], color=COR_SOL, linewidth=2,
            linestyle="--", label=f"Investimento\nR$ {r['custo_total']:,.0f}")

for bar, val in zip(bars, economias_m):
    ax6.text(bar.get_x() + bar.get_width() / 2,
             bar.get_height() + r["custo_total"] * 0.02,
             f"R${val/1000:.0f}k", ha="center", va="bottom",
             color=COR_TEXTO, fontsize=8, fontweight="bold")

ax6.yaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _ : f"R${x/1000:.0f}k"))
ax6.set_xlabel("Horizonte")
ax6.set_ylabel("R$")
ax6.legend(facecolor=COR_FUNDO, edgecolor="#30363D", labelcolor=COR_TEXTO,
           fontsize=8)
ax6.grid(True, axis="y")

# — Rodapé —
fig.text(0.5, 0.01,
        "** Projeção estimada. Valores sujeitos a variações de irradiação, tarifas e condições locais.",
        ha="center", color=COR_CINZA, fontsize=8.5)

output_dir = '/mnt/user-data/outputs/'
os.makedirs(output_dir, exist_ok=True)
plt.savefig(os.path.join(output_dir, "solar_viabilidade.png"),
            dpi=150, bbox_inches="tight", facecolor=COR_FUNDO)
print("  Gráficos salvos em: solar_viabilidade.png")
plt.show()

def upload_pdf():
    uploaded = files.upload()
    if uploaded:
        filename = list(uploaded.keys())[0]
        print(f"Arquivo '{filename}' uploaded com sucesso.")
        return filename
    else:
        print("Nenhum arquivo foi selecionado.")
        return None

```

```

def analisar_pdf_conta_energia(pdf_path):
    text = ""
    try:
        # Tenta extrair texto diretamente com pdfplumber
        with pdfplumber.open(pdf_path) as pdf:
            for page in pdf.pages:
                text += page.extract_text(x_tolerance=2, y_tolerance=2) + "\n"

        # Se pdfplumber não extrair texto ou extrair texto vazio, tenta OCR com pytesseract
        if not text.strip():
            print("pdfplumber não extraiu texto, tentando OCR com pytesseract...")
            # Converte a primeira página do PDF em imagem e usa OCR
            # Adapte para processar mais páginas se necessário
            with pdfplumber.open(pdf_path) as pdf:
                # Certifica-se de que há pelo menos uma página
                if pdf.pages:
                    page_image = pdf.pages[0].to_image(resolution=300).original # Aumenta a
resolução
                    image_bytes = BytesIO()
                    page_image.save(image_bytes, format="PNG")
                    image_bytes.seek(0)
                    text = pytesseract.image_to_string(Image.open(image_bytes), lang='por') #
lang='por' para português
                else:
                    print("O PDF não contém páginas.")

    except Exception as e:
        print(f"Erro ao processar PDF: {e}")
        return None, None

# Debugging: Imprime todo o texto extraído para análise
print("\n--- Texto Completo Extraído do PDF ---")
print(text[:1000]) # Limita para não imprimir PDFs muito grandes
print("-----\n")

consumo_mensal = None
tarifa_kwh = None

# Padrões de expressões regulares para consumo (kWh)
consumo_patterns = [
    r"([0-9]+\.[0-9]*)s*kWh", # Ex: "1234.56 kWh"
    r"Consumo\s*(?:Total|Mês)?\s*:\s*([0-9]+\.[0-9]*)", # Ex: "Consumo total: 1234.56"

```

```

r"TOTAL\s*KWH\s*([0-9]+\.[0-9]*)",      # Ex: "TOTAL KWH 1234.56"
r"Energia Ativa.*?\s*([0-9]+\.[0-9]*)\s*kWh", # Ex: "Energia Ativa 1234 kWh"
r"CONSUMO\s+TOTAL\s+([0-9]+\.[0-9]*)", # Novo: CONSUMO TOTAL 1234.56
r"Quantidade\s+([0-9]+\.[0-9]*)\s+kWh" # Novo: Quantidade 1234.56 kWh
]

print("Tentando extrair Consumo (kWh)...")
for i, pattern in enumerate(consumo_patterns):
    match = re.search(pattern, text, re.IGNORECASE)
    if match:
        consumo_mensal = float(match.group(1).replace(',', '.'))
        print(f" Padrão {i+1} encontrado para consumo: {consumo_mensal} kWh")
        break
    else:
        print(f" Padrão {i+1} de consumo '{pattern}' não encontrado.")

# Padrões de expressões regulares para tarifa (R$/kWh)
tarifa_patterns = [
    r"([0-9]+\.[0-9]*)\s*R\$/kWh",      # Ex: "0.85 R$/kWh"
    r"Tarifa\s*(?:energia|aplicada)?\s*\s*R\$\s*([0-9]+\.[0-9]*)", # Ex: "Tarifa: R$ 0.85"
    r"VALOR\s*KWH\s*R\$\s*([0-9]+\.[0-9]*)", # Ex: "VALOR KWH R$ 0.85"
    r"Tarifa\s+Aplicada\s+([0-9]+\.[0-9]*)", # Novo: Tarifa Aplicada 0.75
    r"TUSD\s+([0-9]+\.[0-9]*)\s*.*?TE\s+([0-9]+\.[0-9]*)" # Novo: para soma de TUSD e TE
]

print("Tentando extrair Tarifa (R$/kWh)...")
for i, pattern in enumerate(tarifa_patterns):
    match = re.search(pattern, text, re.IGNORECASE)
    if match:
        if pattern == r"TUSD\s+([0-9]+\.[0-9]*)\s*.*?TE\s+([0-9]+\.[0-9]*)":
            tUSD = float(match.group(1).replace(',', '.'))
            TE = float(match.group(2).replace(',', '.'))
            tarifa_kwh = tUSD + TE
            print(f" Padrão {i+1} (TUSD+TE) encontrado para tarifa: R$ {tarifa_kwh:.3f}/kWh
(TUSD: {tUSD}, TE: {TE})")
        else:
            tarifa_kwh = float(match.group(1).replace(',', '.'))
            print(f" Padrão {i+1} encontrado para tarifa: R$ {tarifa_kwh:.3f}/kWh")
            break
    else:
        print(f" Padrão {i+1} de tarifa '{pattern}' não encontrado.")

```

Tentativa alternativa: inferir tarifa se o total a pagar e o consumo forem encontrados

```

if tarifa_kwh is None and consumo_mensal is not None:
    print("Tentando inferir tarifa a partir do Total a Pagar e Consumo...")
    total_a_pagar_match = re.search(r"Total\s*a\s*pagar\s*R\s*([0-9]+\.[0-9]*)", text,
re.IGNORECASE)
    if total_a_pagar_match:
        total_a_pagar = float(total_a_pagar_match.group(1).replace(',', '.'))
        if consumo_mensal > 0:
            inferred_tarifa = total_a_pagar / consumo_mensal
            # Considera a tarifa inferida apenas se for um valor razoável
            if 0.1 < inferred_tarifa < 2.0: # Limites razoáveis para tarifa (R$/kWh)
                tarifa_kwh = inferred_tarifa
                print(f"Tarifa inferida: R$ {tarifa_kwh:.3f}/kWh (Total: R$ {total_a_pagar:.2f} /
Consumo: {consumo_mensal:.2f} kWh)")
            else:
                print(f" Tarifa inferida R$ {inferred_tarifa:.3f}/kWh está fora do range razoável
(0.1-2.0).")
        else:
            print(" Consumo mensal é zero, não é possível inferir a tarifa.")
    else:
        print(" Padrão 'Total a pagar' não encontrado para inferir tarifa.")

```

```

return consumo_mensal, tarifa_kwh

```

```

#

```

5. EXECUÇÃO PRINCIPAL MODIFICADA

```

#

```

```

if __name__ == "__main__":
    print("\n" + "=" * 60)
    print(" Opção de Entrada de Dados ")
    print("=" * 60)

    consumo_mensal_kwh = None
    tarifa_kwh = None

    # Loop para garantir uma entrada válida ou extração de PDF
    while consumo_mensal_kwh is None or tarifa_kwh is None:
        escolha = input("Deseja carregar um PDF da conta de energia? (s/n): ").lower()

        if escolha == 's':

```

```

pdf_filename = upload_pdf()
if pdf_filename:
    print("\n" + "=" * 60)
    print(" Analisando PDF... ")
    print("=" * 60)
    consumo_mensal_kwh, tarifa_kwh = analisar_pdf_conta_energia(pdf_filename)
    if consumo_mensal_kwh is not None and tarifa_kwh is not None:
        print(f"Dados extraídos do PDF: Consumo = {consumo_mensal_kwh:.2f} kWh,
Tarifa = R$ {tarifa_kwh:.3f}/kWh")
    else:
        print("\n ⚠ Não foi possível extrair consumo ou tarifa do PDF. Por favor, tente
novamente ou insira manualmente.")
    else:
        print("Nenhum arquivo PDF foi carregado. Inserindo dados manualmente.")
        break # Sai do loop para inserção manual
elif escolha == 'n':
    print("Inserindo dados manualmente.")
    break # Sai do loop para inserção manual
else:
    print("Opção inválida. Por favor, digite 's' para sim ou 'n' para não.")

if consumo_mensal_kwh is None or tarifa_kwh is None: # Se ainda estiver vazio (por falha de
PDF ou escolha 'n')
    print("\n" + "=" * 60)
    print(" Entrada Manual de Dados ")
    print("=" * 60)
    consumo_mensal_kwh = float(input("\n 📊 Consumo médio mensal (kWh): "))
    # Modified line: Replace comma with dot for float conversion
    tarifa_kwh = float(input("\n 💰 Tarifa de energia (R$/kWh) [ex: 0.85]: ").replace(',', '.'))

# As horas de sol e anos de simulação ainda precisam ser manuais ou ter defaults
horas_sol_dia = float(input("\n ☀ Média de horas de sol pleno/dia [ex: 5.0]: "))
anos_simulacao = int(input("\n 📅 Anos de simulação [ex: 25]: "))

dados = (consumo_mensal_kwh, tarifa_kwh, horas_sol_dia, anos_simulacao)
resultado = calcular_sistema(*dados)
exibir_relatorio(resultado)
gerar_graficos(resultado)

```

RESULTADOS

Opção de Entrada de Dados

Deseja carregar um PDF da conta de energia? (s/n): n
Inserindo dados manualmente.

Entrada Manual de Dados



Consumo médio mensal (kWh): 1200



Tarifa de energia (R\$/kWh) [ex: 0.85]: 0.72



Média de horas de sol pleno/dia [ex: 5.0]: 10



Anos de simulação [ex: 25]: 5



RELATÓRIO DE VIABILIDADE



SISTEMA DIMENSIONADO

Potência necessária : 5.00 kWp

Potência instalada : 5.40 kWp

Nº de painéis (450W) : 12 unidades



FINANCEIRO

Investimento total : R\$ 24,300.00

Geração mensal : 1296.0 kWh

Economia 1º mês : R\$ 933.12

Reajuste tarifário : 6% ao ano (projetado)



RETORNO DO INVESTIMENTO

Payback : 26 meses (2.2 anos)



RESULTADO EM 5 ANOS

Total economizado : R\$ 63,121.01

Lucro líquido : R\$ 38,821.01

ROI : 159.8%

CO₂ evitado (~) : 6353 kg



VIÁVEL - o investimento se paga em 2.2 anos!

FINALIDADE

A finalidade deste código é fazer uma análise básica do gasto de energia da propriedade e através de uma avaliação dos parâmetros exigidos mostrar se é viável a instalação de placa solar na propriedade. O código ainda é um protótipo e ainda não consegue ler PDFs de conta de energia corretamente.

